

# **DESIGN AND DEVELOPMENT OF Compliance And Security Auditor**

ISUS ZC425T : Dissertation

by

**PRINCE KUMAR**  
**202219US040**

Dissertation work carried out at

**Unthinkable Solutions, Gurgaon**

Submitted in partial fulfilment of **B.Tech Information  
Systems** degree programme

Under the Supervision of

**Shivakant Sharma**

**Unthinkable Solution, Gurgaon**



**BIRLA INSTITUTE OF TECHNOLOGY & SCIENCE  
PILANI (RAJASTHAN)**

**November 2026**

## ABSTRACT

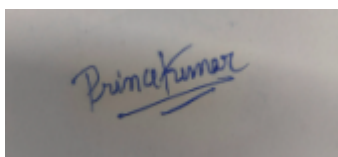
This project proposes an automated platform — Compliance and Network Auditor — designed to detect privacy, security, and regulatory compliance gaps (including GDPR, HIPAA, and ISO 27001) across source code and network-infrastructure.

The solution combines static application security analysis (SAST) with lightweight network auditing to identify flows of PII/PHI, configuration mis-settings, and exposed services early in the development and deployment lifecycle.

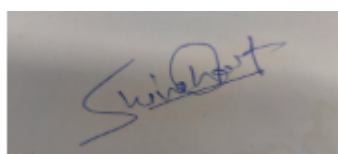
The platform is built with custom rule engine and it also provide extended in depth scanning using external tools or AI models for required scanning, a .NET 8 Web API backend for orchestration and background processing, a React-based dashboard for intuitive reporting, and SQL Server for persistent storage of findings and audit history.

At this mid-semester stage, the custom rule-engine is ready, basic user authentication flow done, scan page and basic findings page are done but rich insightful dashboards and reports are underway, integration of external tools for extended features is pending, and initial network audit modules is pending, while the code scanner has been configured and connected to the CI/CD pipeline. Future scope includes expanding the network auditor to support real-time telemetry, risk scoring per device/service, and automated remediation suggestions — thereby enabling continuous compliance monitoring throughout the full software lifecycle.

Keywords: Compliance Auditor, Network Audit, Privacy, Security, SAST, PII/PHI, Web API, Dashboard, .NET 8, React.



**Signature of the Student, Name: Prince Kumar**



**Signature of the Supervisor, Name: Shivakant Sharma**

**Date: 11/07/2025 , Place: Gurgaon**

## Contents

1. MODULES
2. FUNCTIONAL BLOCK DIAGRAM/ DESCRIPTION
3. MAJOR TECHNICAL SPECIFICATIONS
4. DESIGN CONSIDERATIONS
5. FUTURE PLAN

## 1. MODULES IN COMPLIANCE AND NETWORK AUDITOR (CNA)

The main modules in the **Compliance and Network Auditor (CNA)** are the Custom Analysis Engine with External Integrations (Privado/ML Models), SQL Schema and Code Analyzer, .NET 8 Web API and Background Scan Job Handler, Authentication and OAuth, Role-Based Access Control (RBAC), Rich UI/UX Dashboard built with React, and Insightful Report Generation with subscription management.

---

### SYSTEM / SUB-SYSTEM DESCRIPTION:

The CNA system consists of the following major components namely:

- (a) Custom Analysis Engine and External Integration (Privado/ML Models)
- (b) SQL Schema and Code Analyzer
- (c) .NET 8 Web API and Background Scan Job Handling
- (d) Authentication – Basic and OAuth
- (e) Role-Based Access Control (RBAC)
- (f) Rich UI/UX and Dashboard built with React
- (g) Insightful Report Generation and Subscription Handling

The description of each of the major components is explained below:

#### a) Custom Analysis Engine and External Integration (Privado/ML Models):

The Custom Analysis Engine serves as the core intelligence layer of the CNA platform. It integrates the open-source **Privado** privacy scanner with extended rule sets and custom machine learning models to detect security, privacy, and compliance gaps in source code. This module identifies PII/PHI data flows, insecure data storage, hardcoded credentials, and violations mapped to standards such as GDPR, HIPAA, and ISO 27001. The system supports modular integration with external analysis tools and ML-based anomaly detectors, allowing future extension for data classification and pattern learning. The engine runs autonomously or on-demand through the orchestrator, ensuring early and automated detection during code commits or deployment.

#### b) SQL Schema and Code Analyzer:

This module defines and manages the **structured database schema** and performs **semantic code analysis** to correlate findings from multiple sources. It stores scan metadata, rule definitions, detected issues, audit history, and remediation status. The Code Analyzer complements the static engine by parsing project repositories, identifying high-risk areas (controllers, data-access layers), and linking code-level issues with corresponding database entities. SQL Server is used for persistence, ensuring relational integrity and efficient query performance. The schema supports fine-grained audit trails, report generation, and role-based data segregation to maintain compliance integrity.

#### b) SQL Schema and Code Analyzer:

This module defines and manages the **structured database schema** and performs **semantic code analysis** to correlate findings from multiple sources. It stores scan metadata, rule definitions, detected issues, audit history, and remediation status. The Code Analyzer complements the static engine by parsing project repositories, identifying high-risk areas (controllers, data-access layers), and linking code-level issues with corresponding database entities. SQL Server is used for persistence, ensuring relational integrity and efficient query performance. The schema supports

fine-grained audit trails, report generation, and role-based data segregation to maintain compliance integrity.

---

#### **c) .NET 8 Web API and Background Scan Job Handling:**

The **.NET 8 Web API** acts as the backbone of the CNA platform, orchestrating all operations between modules. It exposes secure REST endpoints for scan initiation, scheduling, and retrieving results. Background job handling is implemented using hosted services and queues to manage large-scale or asynchronous scanning workloads. The orchestrator handles task prioritization, load balancing, and result aggregation from the analysis engine and network audits. Error handling, logging, and retry mechanisms ensure resilience and stability during continuous scan operations. The API also acts as an integration hub for CI/CD pipelines, enabling seamless automation of compliance checks during the software development lifecycle.

---

#### **d) Authentication – Basic and OAuth:**

The Authentication module provides a secure identity layer supporting both **Basic Authentication** and **OAuth 2.0** standards. Basic authentication is used for local or internal deployments, while OAuth enables integration with external identity providers such as Google, Microsoft, or enterprise SSO systems. Tokens are issued using **JWT (JSON Web Token)** with configurable expiration and refresh mechanisms. The authentication flow ensures secure access to all CNA endpoints, protects sensitive compliance data, and logs all access attempts for audit purposes. Future versions will support MFA and adaptive login policies based on user risk level.

---

#### **e) Role-Based Access Control (RBAC):**

The **RBAC module** enforces permission-based access management across all user types. Three main roles are defined — **Administrator**, **Auditor**, and **Developer** — each with predefined privileges and scopes. Administrators manage users, policies, and global rules; Auditors review and close findings; Developers access only assigned projects or findings. The RBAC layer integrates directly with authentication and database schemas to ensure separation of duties and data confidentiality. All user activities are logged to maintain a complete and traceable audit trail, meeting the accountability requirements of ISO 27001 and GDPR Article 30.

---

#### **f) Rich UI/UX and Dashboard built with React:**

The **React-based Dashboard** provides a modern, interactive, and responsive interface for all users. It enables easy submission of code or network scan requests, visualization of compliance metrics, and tracking of remediation activities. The dashboard offers drill-down views into privacy violations, data flow paths, and audit progress. Real-time updates are powered through API calls, and visualization components display trends, severity distribution, and compliance scoring. The user interface follows accessibility and design best practices to ensure clarity and usability for both technical and non-technical users.

---

#### **g) Insightful Report Generation and Subscription Handling:**

The Report Generation module compiles audit results, compliance summaries, and historical performance metrics into exportable formats (PDF, CSV). Each report includes detailed findings, severity categorization, regulation mapping, and recommended corrective actions. Users can subscribe to **automated compliance reports** via email or web notifications at defined intervals (daily, weekly, or monthly). The system supports scheduled report generation, digital signatures for authenticity, and delivery tracking for audit evidence. This module ensures transparency, accountability, and ease of communication across compliance and development teams.

---

Together, these modules create a unified ecosystem for **automated privacy and network compliance auditing**, integrating code analysis, intelligent orchestration, and actionable reporting to ensure organizations continuously adhere to GDPR, HIPAA, and ISO 27001 standards throughout the software lifecycle.

## **DETAILED FUNCTIONAL NOTES**

### **1. MESSAGE / JOB BUS**

The Task Queue is the system's control plane: scan requests are accepted by the .NET API and enqueued with a job manifest (targets, scan profile, credentials, schedule). The queue supports priority levels (ad-hoc vs scheduled), visibility timeouts, and retries. Workers subscribe to job topics and acknowledge completion or push back for retries. This decouples UI responsiveness from long-running scans and enables horizontal scale.

### **2. .NET 8 WEB API**

The API handles authentication, job orchestration, policy evaluation, and result aggregation. It exposes endpoints for submitting jobs (repo URL, network CIDR, DB credentials), retrieving results, managing rulesets and RBAC configuration. The orchestrator spawns background jobs, coordinates worker scaling, and records provenance (scan timestamp, tool versions, rule versions) into SQL Server.

### **3. STATIC ANALYSIS & ML INTEGRATION (Analysis Engine)**

This engine integrates **Privado** extended with custom rule sets and optional ML models for improved detection (NER models for PII/PHI, classifier models for risky patterns). It performs source parsing, taint/data-flow analysis (source→sink), and produces structured findings: location (file/line), flow trace, severity, suggested remediation, and regulation mapping. The engine can run CLI style in CI or as a worker invoked by the orchestrator.

### **4. SQL SCHEMA & CODE ANALYZER**

This module analyses database schemas and application data-access code to identify where sensitive fields are stored or transformed (e.g., cleartext storage, missing encryption, no retention policy). It correlates DB columns to code-level sinks/sources to provide holistic findings (e.g., controller X writes PII to table Y).

### **5. RESULT ENRICHER & CORRELATOR**

After raw analysis, the Enricher augments findings with context — mapping a finding to GDPR/HIPAA/ISO controls, adding evidence (stack traces, headers, sample DB rows), and cross-linking related findings across code and network (e.g., service A exposed on port 8080 corresponds to an API sink found in repo B).

### **6. REPORT GENERATION & SUBSCRIPTION**

Periodic or on-demand reports are compiled into exportable bundles (PDF/CSV) including finding lists, remediation evidence, and history. Users can subscribe to scheduled reports; the subscription service triggers the Report Worker, sends emails or webhook notifications, and stores delivery records for audit.

### **7. AUTHENTICATION & RBAC**

Auth module issues JWT tokens (or integrates with OAuth/OIDC). RBAC enforces least privilege across REST and UI surfaces; all sensitive actions and accesses are logged with user identity for non-repudiation.

---

## **Functional Flow**

1. User or CI webhook submits a job to the .NET API (target + profile).
2. API authenticates user, validates job, writes metadata to SQL and enqueues job on Task Queue.
3. Appropriate worker picks up the job (Static Analysis / Network / SQL Analyzer).

4. Worker runs analysis calls custom rule engine or for extended scope ( calls Privado / ML models / agent), deposits artifacts to blob store, writes findings to SQL.
5. Enricher correlates and maps findings to compliance controls; severity and remediation hints added.
6. Report Worker compiles reports and subscription triggers send notifications.
7. UI polls or receives WebSocket updates and displays findings; RBAC controls visibility.

## 1. MAJOR TECHNICAL SPECIFICATIONS

| SL No. | Technical Parameter                | Specification / Description                                                   |
|--------|------------------------------------|-------------------------------------------------------------------------------|
| 1.     | <b>System Type</b>                 | Web-Based Compliance and Network Auditing Platform                            |
| 2.     | <b>Core Frameworks Used</b>        | .NET 8 Web API (Backend), React.js (Frontend)                                 |
| 3.     | <b>Programming Languages</b>       | C#, JavaScript (ES6+), SQL, Python (for ML integrations)                      |
| 4.     | <b>Processor (Server)</b>          | Intel Xeon Silver / Core i7 12th Gen equivalent (Multi-core)                  |
| 5.     | <b>Processor Frequency</b>         | Base: 2.4 GHz, Turbo Boost: up to 4.2 GHz                                     |
| 6.     | <b>System Memory (RAM)</b>         | 16 GB DDR4, scalable to 32 GB (for server execution)                          |
| 7.     | <b>Storage</b>                     | 512 GB SSD (System), extendable storage for scan data (1 TB recommended)      |
| 8.     | <b>Database</b>                    | Microsoft SQL Server 2022 (Persistent storage for findings and users)         |
| 9.     | <b>Operating System</b>            | Windows Server 2022 / Linux (Ubuntu 22.04 LTS) supported                      |
| 10.    | <b>Frontend Framework</b>          | React.js with Material UI, Axios, and Chart.js for visualization              |
| 11.    | <b>Backend Framework</b>           | ASP.NET Core 8.0 Web API with Background Services and Task Queue              |
| 12.    | <b>Analysis Engine</b>             | Custom Analysis Engine integrated with Privado and ML-based detectors         |
| 13.    | <b>External Integrations</b>       | Privado API, OAuth 2.0, CI/CD Webhooks (GitHub/GitLab), Mailer Service        |
| 14.    | <b>Authentication</b>              | Basic Authentication & OAuth 2.0 with JWT Tokens                              |
| 15.    | <b>Access Control</b>              | Role-Based Access Control (RBAC): Admin, Auditor, Developer roles             |
| 16.    | <b>User Interface</b>              | Responsive Web Dashboard (React.js) with real-time updates                    |
| 17.    | <b>Reporting</b>                   | Automated and On-Demand Report Generation (PDF/CSV/Excel formats)             |
| 18.    | <b>Communication Interface</b>     | REST API, WebSocket (SignalR) for live updates                                |
| 19.    | <b>Job Handling</b>                | Message Queue-based background job processor for parallel scans               |
| 20.    | <b>Browser Support</b>             | Chrome (v120+), Edge (v120+), Firefox (v115+), Safari (v17+)                  |
| 21.    | <b>Deployment Mode</b>             | Docker-based container deployment or standalone server setup                  |
| 22.    | <b>Reporting Engine</b>            | Integrated Report Generator and Subscription Scheduler                        |
| 23.    | <b>Network Dependency</b>          | Outbound Internet access required only for external rule updates and ML scans |
| 24.    | <b>Power / Resource Management</b> | Lightweight background service model; CPU throttling for continuous operation |

| SL No. | Technical Parameter        | Specification / Description                                              |
|--------|----------------------------|--------------------------------------------------------------------------|
| 25.    | <b>Audit and Logs</b>      | Persistent, immutable audit logging with timestamp and user traceability |
| 26.    | <b>Output Formats</b>      | JSON, CSV, PDF, HTML dashboard summaries                                 |
| 27.    | <b>Compliance Coverage</b> | GDPR, HIPAA, ISO 27001, ISO 27701                                        |

#### 4. DESIGN CONSIDERATIONS:

- Privacy-first scanning: minimize storage of raw PII; store only findings and metadata needed for evidence.
- Scalability: background job queues and modular scanners to scale across multiple workers.
- Security: secure APIs, input validation, rate limiting, and encrypted storage for sensitive metadata.
- Extensibility: rule-based scanner extensions allowing new compliance rules to be added without core changes.
- Reliability: repeatable scans with history, insightful reports
- Usability: straightforward UI for developers and auditors with clear remediation steps and exportable evidence.



## 5. FUTURE PLAN:

| SI No | Phases                 | Work to be done                                                                 | Status    |
|-------|------------------------|---------------------------------------------------------------------------------|-----------|
| 1     | Dissertation outline   | Literature review and prepare Dissertation outline                              | COMPLETED |
| 2     | Design and Development | Design & Development Activity                                                   | PENDING   |
| 3     | Testing                | Software Testing                                                                | PENDING   |
| 4     | Dissertation Review    | Submit dissertation to supervisor & Additional Examiner for review and feedback | PENDING   |
| 5     | Submission             | Final Review and submission of dissertation                                     | PENDING   |

## 6. ABBREVIATIONS

| Abbreviation        | Full Form / Description                                           |
|---------------------|-------------------------------------------------------------------|
| <b>AI</b>           | Artificial Intelligence                                           |
| <b>API</b>          | Application Programming Interface                                 |
| <b>ASP.NET</b>      | Active Server Pages .NET (Web Application Framework by Microsoft) |
| <b>CAPA</b>         | Corrective and Preventive Action                                  |
| <b>CI/CD</b>        | Continuous Integration / Continuous Deployment                    |
| <b>CPU</b>          | Central Processing Unit                                           |
| <b>CNA</b>          | Compliance and Network Auditor                                    |
| <b>DB</b>           | Database                                                          |
| <b>GDPR</b>         | General Data Protection Regulation                                |
| <b>HIPAA</b>        | Health Insurance Portability and Accountability Act               |
| <b>HTTP / HTTPS</b> | HyperText Transfer Protocol / Secure HTTP                         |
| <b>I/O</b>          | Input / Output                                                    |
| <b>ISO</b>          | International Organization for Standardization                    |
| <b>JSON</b>         | JavaScript Object Notation                                        |
| <b>JWT</b>          | JSON Web Token                                                    |
| <b>LTS</b>          | Long-Term Support (for OS or frameworks)                          |
| <b>ML</b>           | Machine Learning                                                  |
| <b>NLP</b>          | Natural Language Processing                                       |
| <b>OAuth</b>        | Open Authorization (Access Delegation Protocol)                   |
| <b>OIDC</b>         | OpenID Connect                                                    |
| <b>PII</b>          | Personally Identifiable Information                               |
| <b>PHI</b>          | Protected Health Information                                      |
| <b>RBAC</b>         | Role-Based Access Control                                         |
| <b>REST</b>         | Representational State Transfer                                   |
| <b>SAST</b>         | Static Application Security Testing                               |
| <b>SQL</b>          | Structured Query Language                                         |
| <b>SSO</b>          | Single Sign-On                                                    |
| <b>TLS</b>          | Transport Layer Security                                          |
| <b>UI / UX</b>      | User Interface / User Experience                                  |
| <b>VM</b>           | Virtual Machine                                                   |
| <b>WS</b>           | WebSocket                                                         |
| <b>YAML</b>         | YAML Ain't Markup Language (used for rule configuration)          |
| <b>.NET</b>         | Microsoft's Cross-Platform Framework for Application Development  |